ALA-2

Q1. What are Bitwise Operators? Explain their purpose in programming.

Answer:
Bitwise operators are special operators used to perform operations on numbers at the **binary (bit) level**. Computers store numbers in binary form (0 and 1), so bitwise operators allow programmers to directly manipulate these bits.

The main purpose of bitwise operators is to perform **fast calculations**, **data manipulation**, and **memory-efficient operations**. They are widely used in system programming, networking, encryption, and embedded systems.

For example:
If a = 5 and b = 3
Binary form:
5 = 101
3 = 011

Using AND operator:
5 & 3 = 1

Q2. Explain the Bitwise AND (&) operator with syntax and example.

Answer:
The Bitwise AND operator compares each bit of two numbers. If both bits are **1**, the result is **1**; otherwise, it is **0**.

Syntax:

result = a & b

Example:
a = 5
b = 3

Binary:
5 = 101
3 = 011

Result:
101
011

001 = 1

Q3. Explain the Bitwise OR (|) operator with syntax and example.

Answer:
The Bitwise OR operator compares each bit of two numbers. If at least **one bit is 1**, the result becomes **1**.

Syntax:

result = a | b

Example:
a = 6
b = 2

Binary:
6 = 110
2 = 010

Result:
110
010

110 = 6

Q4. Explain the Bitwise XOR (^) operator with example.

Answer:
The Bitwise XOR (Exclusive OR) operator compares bits of two numbers. It gives **1 only when bits are different**, otherwise **0**.

Syntax:

result = a ^ b

Example:
a = 5
b = 3

Binary:
5 = 101
3 = 011

Result:
101
011

110 = 6

Q5. Explain the Bitwise NOT (~) operator with example.

Answer:
The Bitwise NOT operator changes **1 to 0** and **0 to 1**. It reverses all bits of a number.

Syntax:

result = ~a

Example:
a = 5

Binary of 5 = 00000101
After NOT = 11111010

So,

print(~5)

Output: **-6**

The NOT operator is used in **complement operations** and **bit inversion tasks**

Q6. Explain the Left Shift (<<) operator with example.

Answer:
The Left Shift operator moves bits to the **left** by a specified number of positions. Each shift to the left multiplies the number by **2**.

Syntax:

result = a << n

Example:
a = 4

Binary:
4 = 100

After shifting left by 1:
100 → 1000

Result:

print(4 << 1)

Output: **8**

Left Shift is used in **fast multiplication** operations.

Q7. Explain the Right Shift (>>) operator with example.

Answer:
The Right Shift operator moves bits to the **right**. Each shift to the right divides the number by **2**.

Syntax:

result = a >> n

Example:
a = 8

Binary:
8 = 1000

After shifting right by 2:
1000 → 0010

Result:

print(8 >> 2)

Output: **2**

Right Shift is used in **fast division** operations.

Q8. Write a program to demonstrate Bitwise AND, OR and XOR operators.

Answer:

```
a = 5
b = 3

print("AND:", a & b)
print("OR:", a | b)
print("XOR:", a ^ b)
```

Output:

```
AND: 1
OR: 7
XOR: 6
```

This program demonstrates how three common bitwise operators work together.

Q9. Differentiate between Bitwise Operators and Logical Operators.

Answer:

Bitwise Operators	**Logical Operators**
Work on binary bits	Work on Boolean values
Used for low-level operations	Used for conditions
Examples: &, |, ^	Examples: and, or, not
Faster operations	Used in decision making

Example:

```
print(5 & 3)   # Bitwise
print(True and False)  # Logical
```

Q10. Explain real-life applications of Bitwise Operators.

Answer:
Bitwise operators are widely used in many real-life programming applications.

Some important applications include:

1. **Data Compression**
   Bitwise operations help reduce file size efficiently.
2. **Encryption and Security**
   XOR operator is commonly used in encryption algorithms.
3. **Graphics Programming**
   Used to manipulate pixel values.
4. **Network Programming**
   Used in IP address calculations.
5. **Memory Optimization**
   Helps store multiple values in a single variable.

Thus, bitwise operators are very useful in **system-level programming**, **networking**, and **embedded systems**.